

DOJOBID — App Build Specification

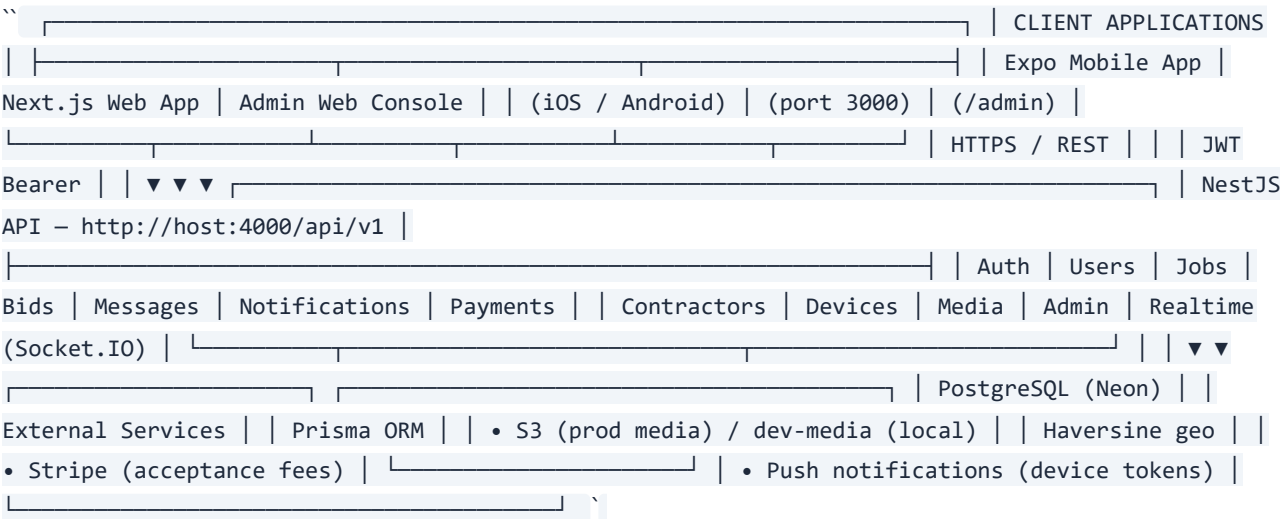
Product: DOJOBID — cross-platform marketplace connecting homeowners with local contractors
Repository: ContractorBidder (npm monorepo) **Version:** 0.1.0 **Document date:** June 7, 2026

1. Executive Summary

DOJOBID is a two-sided marketplace where **homeowners** post home-improvement jobs and **contractors** discover nearby work, submit bids, and communicate after a bid is accepted. The platform enforces a **privacy-first location model**: contractors see only coarse geography (ZIP code and ~1 km grid coordinates) until a bid is accepted, at which point the full address and precise coordinates are revealed. The system is implemented as a **TypeScript monorepo** with three client surfaces and one backend: SurfaceTechnologyPurpose __TABLE_SEP__ APINestJS 10 + Prisma 6 + PostgreSQL (Neon)REST API, auth, business logic, media, payments WebNext.js 14 (App Router)Browser client for homeowners and contractors MobileExpo SDK 54 + React Navigation 7iOS/Android native experience Shared @contractor-bidder/types , config , ui Shared types and UI primitives Real-time updates use **Socket.IO** on the /realtime namespace. Authentication uses **JWT** (access + refresh tokens) with **Argon2id** password hashing.

2. System Architecture

2.1 High-Level Diagram



2.2 Monorepo Layout

```
ContractorBidder/
├── apps/
│   ├── api/ NestJS backend, Prisma schema, seed data
│   └── web/ Next.js 14 web application
├── mobile/ Expo app (outside npm workspaces)
└── packages/
```

types/ Shared TypeScript API contract types | └─ config/ Shared ESLint/TS config | └─ ui/ Shared React UI components | └─ docs/ Documentation (this file) | └─ package.json Root workspace scripts` **Note:** apps/mobile is managed with its own package.json and linked via npm --prefix apps/mobile because Expo tooling does not fully participate in the root npm workspaces.

2.3 Root Scripts

CommandDescription
__TABLE_SEP__ npm run api Start NestJS API in dev mode (port 4000) npm run web Start Next.js dev server (port 3000) npm run mobile Start Expo Go with LAN (expo start --go --lan) npm run db:generate Regenerate Prisma client npm run db:migrate Run Prisma migrations npm run db:seed Seed demo users and jobs npm run build Build all workspaces npm run typecheck Type-check all workspaces

3. Technology Stack

3.1 Backend (apps/api)

LayerChoiceNotes __TABLE_SEP__ FrameworkNestJS 10Modular controllers, guards, pipes ORMPrisma 6Rust-free prisma-client generator + @prisma/adaptor-pg DatabasePostgreSQL on NeonCloud-hosted; works on Windows ARM64 AuthJWT + Argon2idAccess token (short) + refresh token (rotating) Validationclass-validatorGlobal ValidationPipe with whitelist RealtimeSocket.IO /realtime gateway for live events MediaS3 presigned URLsDev fallback: local .dev-media/ served by API PaymentsStripeWebhook at POST /payments/webhook/stripe **API base URL:** http://localhost:4000/api/v1 (dev) **Global prefix:** /api with URI versioning default v1

3.2 Web (apps/web)

LayerChoice __TABLE_SEP__ FrameworkNext.js 14 App Router StylingTailwind CSS AuthJWT stored client-side, attached to API requests **Routes:** PathPurpose __TABLE_SEP__ / Landing / home /login Login and registration /jobs Job search (contractors) /jobs/new Post a job (homeowners) /jobs/[id] Job detail, bids, messaging /my-jobs Homeowner's posted jobs /admin Admin dashboard

3.3 Mobile (apps/mobile)

LayerChoice __TABLE_SEP__ FrameworkExpo SDK 54 NavigationReact Navigation 7 (bottom tabs + stack) Locationexpo-location (GPS + geocoding) Mediaexpo-image-picker + presigned upload Dev clientExpo Go via npm run start:go **Screens:** ScreenRoleDescription __TABLE_SEP__ LoginScreenAllLogin / register MyJobsScreenHomeownerPosted jobs list PostJobScreenHomeownerCreate job with address geocoding + photos FindJobsScreenContractorRadius search, trade filters, pull-to-refresh JobDetailScreenAllJob details, bid, accept, messaging ActivityScreenAllNotifications feed ProfileScreenAllUser profile; contractor service area + trades **Navigation rules:** The Find tab is visible only to CONTRACTOR and ADMIN roles. Homeowners land on My Jobs.

3.4 Shared Packages

1. **@contractor-bidder/types** – DTOs, enums, serialized view types shared by web and mobile

1. **@contractor-bidder/config** – ESLint and TypeScript base configs

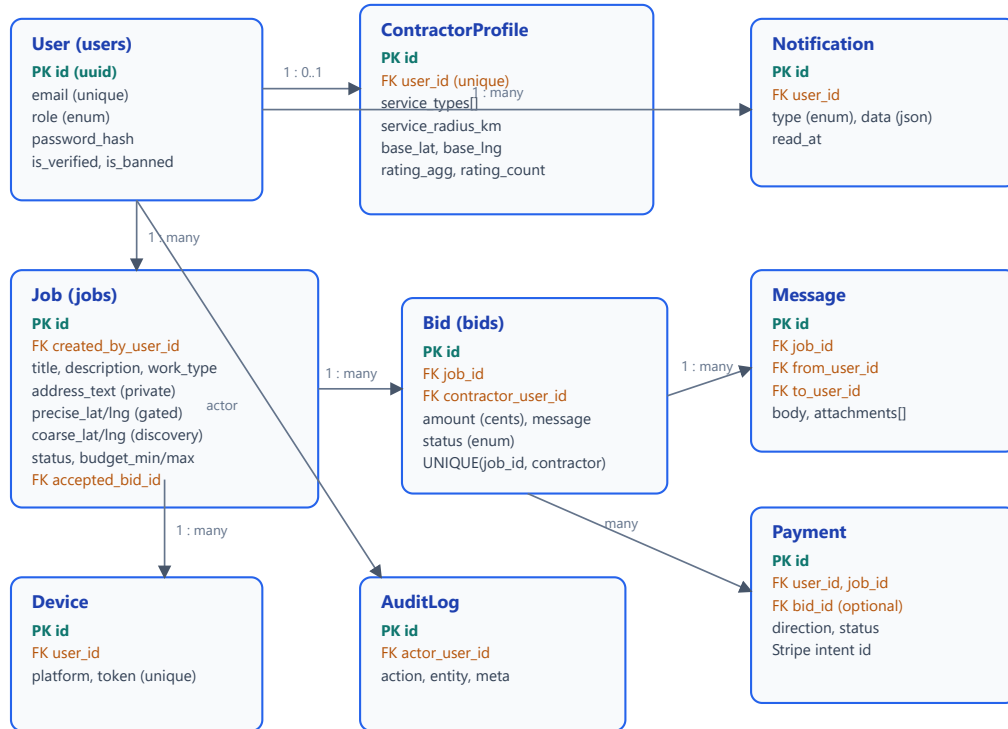
1. **@contractor-bidder/ui** – Reusable React components
-

4. Data Model

4.1 Entity Relationship Diagram

DOJOBID Entity Relationship Diagram

PostgreSQL schema (Prisma) ♦ 10 tables ♦ 11 enums



Privacy: Job.address_text + precise_lat/lng revealed only to owner and accepted contractor (after optional \$1 acceptance fees).
Discovery uses coarse_lat/lng (~1 km grid) and postal_code derived from address for non-owners.

4.2 Tables Overview

The PostgreSQL schema defines **10 models** and **11 enums**. All primary keys are UUIDs. ##### User (users) Central identity record. Supports roles HOMEOWNER, CONTRACTOR, and ADMIN. Password auth uses Argon2id; OAuth providers (GOOGLE, APPLE) are modeled but not yet wired in the UI. ColumnTypeDescription __TABLE_SEP__ idUUIDPrimary key emailString(unique)Login identifier password_hashString?Argon2id hash (password auth only) roleRole enumHOMEOWNER, CONTRACTOR, ADMIN first_name, last_nameStringDisplay name phoneString?Optional contact is_verifiedBooleanEmail verification flag is_bannedBooleanAdmin ban flag ##### ContractorProfile (contractor_profiles) One-to-one with User (contractor role). Stores service capabilities and search radius. ColumnTypeDescription __TABLE_SEP__ user_idUUID (unique FK)Links to User service_typesString[]Trade filters (e.g. electrical, plumbing) service_radius_kmFloatDefault 25 km search radius base_lat, base_lngFloat?Service area center (GPS) rating_agg, rating_countFloat, IntAggregated rating (future) ##### Job (jobs) A homeowner-posted work request. Location fields implement the privacy model. ColumnTypeDescription __TABLE_SEP__ created_by_user_idUUID FKHomeowner who posted title, descriptionStringJob content work_typeStringOne of seven trade categories address_textStringPrivate – full street address precise_lat, precise_lngFloat?Private

– exact coordinates coarse_lat, coarse_lngFloatPublic – snapped to ~1 km grid photosString[]Media URLs budget_min, budget_maxInt?Budget range in cents statusJobStatusOPEN, UNDER_REVIEW, CLOSED, CANCELLED, AWARDED accepted_bid_idUUID? FKWinning bid reference ##### Bid (bids) Contractor offer on a job. Unique constraint prevents duplicate bids per contractor per job. ColumnTypeDescription __TABLE_SEP__ job_idUUID FKTarget job contractor_user_idUUID FKBidding contractor amountIntBid amount in cents messageString?Optional pitch statusBidStatusPENDING, WITHDRAWN, DECLINED, ACCEPTED, EXPIRED ##### Message (messages) Direct messages between users on a job thread. ColumnTypeDescription __TABLE_SEP__ job_idUUID FKConversation context from_user_id, to_user_idUUID FKParticipants bodyStringMessage text attachmentsString[]Optional media URLs visibilityMessageVisibilityALL_BIDDERS or DIRECT ##### Notification (notifications) In-app notification records with typed payloads stored as JSON. TypeTrigger __TABLE_SEP__ JOB_MATCHNew job matches contractor filters NEW_BIDHomeowner receives a bid BID_ACCEPTEDContractor's bid accepted MESSAGENew message received PAYMENT_REQUIREDAcceptance fee due ##### Payment (payments) Stripe-backed acceptance fees (\$1 homeowner fee, \$1 contractor fee model). ColumnTypeDescription __TABLE_SEP__ directionPaymentDirectionHOMEOWNER_ACCEPT_FEE or CONTRACTOR_ACCEPT_FEE statusPaymentStatusPENDING, SUCCEEDED, FAILED, CANCELLED provider_payment_intent_idString?Stripe PaymentIntent ID ##### Device (devices) Push notification tokens per platform (IOS, ANDROID, WEB). ##### AuditLog (audit_logs) Admin and system action trail with JSON metadata.

4.3 Enums Reference

EnumValues __TABLE_SEP__ RoleHOMEOWNER, CONTRACTOR, ADMIN AuthProviderPASSWORD, GOOGLE, APPLE LocationPrecisionPRECISE, COARSE JobStatusOPEN, UNDER_REVIEW, CLOSED, CANCELLED, AWARDED BidStatusPENDING, WITHDRAWN, DECLINED, ACCEPTED, EXPIRED MessageVisibilityALL_BIDDERS, DIRECT NotificationTypeJOB_MATCH, NEW_BID, BID_ACCEPTED, MESSAGE, PAYMENT_REQUIRED PaymentDirectionHOMEOWNER_ACCEPT_FEE, CONTRACTOR_ACCEPT_FEE PaymentStatusPENDING, SUCCEEDED, FAILED, CANCELLED PaymentProviderSTRIPE DevicePlatformIOS, ANDROID, WEB

4.4 Work Types

Seven standardized trade categories used across API validation, search filters, and UI labels: ValueLabel __TABLE_SEP__ electricalElectrical plumbingPlumbing landscapingLandscaping haulingHauling carpentryCarpentry handymanHandyman otherOther

5. Privacy and Location Model

DOJOBID separates **discovery data** from **private location data**:

1. **On job creation:** The API geocodes the homeowner's address into `precise_lat/lng` and computes `coarse_lat/lng` by snapping coordinates to a ~1 km grid.
1. **For contractors browsing jobs:** The serialized job view exposes `postalCode` (extracted from `address_text`) and coarse coordinates – **not** the street address or precise coordinates.
1. **After bid acceptance:** The job owner and accepted contractor receive the full `addressText`, `preciseLat`, and `preciseLng` in the API response.
1. **Contractor search:** Uses Haversine distance from the contractor's `base_lat/lng` (service area) against job `coarse_lat/lng`, filtered by `service_radius_km` and matching `work_type`.

Future upgrade: The Prisma schema includes notes for migrating to PostGIS `geography(Point,4326)` with `ST_DWithin` for native radius queries. Current implementation uses plain `Float` columns and Haversine SQL.

6. API Reference

All endpoints are prefixed with `/api/v1`. Authenticated routes require `Authorization: Bearer` unless noted.

6.1 Health and Feature Flags

Method	Path	Auth	Description
GET	/health	No	Health check
GET	/flags	No	Feature flag snapshot

6.2 Authentication

MethodPathAuthDescription __TABLE_SEP__ POST /auth/register NoRegister homeowner or contractor
POST /auth/login NoLogin, returns JWT pair POST /auth/refresh NoRotate refresh token
POST /auth/logout YesInvalidate refresh token

6.3 Users

MethodPathAuthDescription __TABLE_SEP__ GET /users/me YesCurrent user profile
PATCH /users/me YesUpdate profile fields

6.4 Contractors

MethodPathAuthDescription __TABLE_SEP__ POST /contractors/profile YesCreate contractor profile
PATCH /contractors/profile YesUpdate trades, radius, location GET /contractors/:userId/profile YesRead
contractor profile

6.5 Jobs

MethodPathAuthDescription __TABLE_SEP__ POST /jobs YesCreate job (homeowner)
GET /jobs/search YesRadius + trade search (contractor) GET /jobs/mine YesList jobs created by
current user GET /jobs/:id YesJob detail (privacy-aware serialization) POST /jobs/:id/close YesClose
job to new bids POST /jobs/:id/cancel YesCancel job

6.6 Bids

MethodPathAuthDescription __TABLE_SEP__ POST /jobs/:jobId/bids YesSubmit bid (contractor)
GET /jobs/:jobId/bids YesList bids on job PATCH /bids/:id YesUpdate or withdraw bid
POST /bids/:id/accept YesAccept bid (homeowner)

6.7 Messages

MethodPathAuthDescription __TABLE_SEP__ POST /messages YesSend message on a job GET /messages?
jobId= YesList messages for job

6.8 Notifications

MethodPathAuthDescription __TABLE_SEP__ GET /notifications YesList user notifications
POST /notifications/mark-read YesMark notifications read

6.9 Payments

MethodPathAuthDescription __TABLE_SEP__ POST /payments/session YesCreate Stripe checkout session
GET /payments/:id YesPayment status POST /payments/webhook/stripe Stripe sigStripe webhook
handler

6.10 Media

MethodPathAuthDescription __TABLE_SEP__ POST /media/sign-upload YesGet presigned PUT URL for
photo PUT /dev-media/* NoDev-only: receive uploaded bytes GET /dev-media/* NoDev-only: serve

stored media

6.11 Devices

Method	Path	Auth	Description
POST	/devices	Yes	Register push token
DELETE	/devices/:token	Yes	Unregister push token

6.12 Admin

Method	Path	Auth	Description
GET	/admin/jobs	Admin	List all jobs
GET	/admin/users	Admin	List all users
POST	/admin/flags/ban-user	Admin	Ban user
POST	/admin/flags/unban-user	Admin	Unban user

7. Core Business Flows

7.1 Registration and Onboarding

```
` Homeowner Contractor | | | POST /auth/register | POST /auth/register (role=CONTRACTOR) |  
(role=HOMEOWNER) | | | POST /contractors/profile | | (trades, radius, GPS base location) |  
Ready to post jobs | Ready to search jobs `
```

7.2 Post a Job (Homeowner)

1. Homeowner enters title, description, work type, budget, desired dates.

1. Address is entered as text; client geocodes via expo-location (mobile) or equivalent (web).

1. Photos selected; client calls POST /media/sign-upload , PUTs bytes to signed URL.

1. POST /jobs creates record; API stores precise + coarse coordinates.

1. Matching contractors receive JOB_MATCH notifications.

7.3 Find and Bid (Contractor)

1. Contractor opens Find Jobs; app reads GPS for current location.

1. GET /jobs/search?lat=&lng=&radiusKm=&workType= returns nearby open jobs.

1. Job cards show title, trade, budget, ZIP code (not full address).

1. Contractor opens detail, submits POST /jobs/jobId/bids with amount and message.

1. Homeowner receives NEW_BID notification.

7.4 Accept Bid and Reveal Location

1. Homeowner reviews bids on job detail screen.

1. POST /bids/:id/accept marks bid ACCEPTED, job status AWARDED.

1. Optional: acceptance triggers Stripe payment session for \$1 fees.

1. API response now includes full address and precise coordinates for both parties.

1. Contractor receives BID_ACCEPTED notification; messaging unlocked.

7.5 Messaging

1. Either party sends POST /messages with jobId , toUserId , and body.

1. Recipient receives MESSAGE notification.

1. Real-time delivery via Socket.IO /realtime namespace (when connected).

7.6 Job Lifecycle States

OPEN → UNDER_REVIEW → AWARDED | | → CLOSED → (work complete – manual close) → CANCELLED

8. Authentication and Security

ConcernImplementation __TABLE_SEP__ Password storageArgon2id hashing Session tokensJWT access (15 min) + refresh (7 days, rotating) AuthorizationNestJS guards: @Roles() , ownership checks on jobs/bids Input validationGlobal ValidationPipe with whitelist + forbidNonWhitelisted CORSConfigurable via API_CORS_ORIGIN (comma-separated origins) Stripe webhooksRaw body preserved; signature verification Admin actionsAuditLog entries for ban/unban and sensitive operations Banned users is_banned flag blocks authenticated requests

9. Media Upload Pipeline

Production (S3)

1. Client requests POST /media/sign-upload with filename and content type.

1. API returns presigned S3 PUT URL and final fileUrl .

1. Client PUTs image bytes directly to S3.

1. Job stores fileUrl in photos[] .

Development (Local Dev-Media)

When S3 credentials are absent, the API uses a local .dev-media/ directory:

1. sign-upload returns a PUT URL pointing to /api/v1/dev-media/ .

1. Raw body middleware accepts up to 12 MB on the dev-media route.

1. URLs use the request Host header so mobile devices on LAN can fetch images.

1. Mobile keeps a local previewUri for immediate thumbnail display.

10. Realtime Events

Socket.IO gateway mounted at `/realtime`. Clients authenticate with JWT on connection. Events include new bids, messages, and notification updates. Mobile and web clients subscribe per user session.

11. Environment Configuration

11.1 API (`apps/api/.env`)

Variable	Purpose
<code>__TABLE_SEP__</code>	DATABASE_URL Neon PostgreSQL connection string
<code>JWT_ACCESS_SECRET</code>	Access token signing key
<code>JWT_REFRESH_SECRET</code>	Refresh token signing key
<code>JWT_ACCESS_EXPIRES</code>	Access token TTL (e.g. 15m)
<code>JWT_REFRESH_EXPIRES</code>	Refresh token TTL (e.g. 7d)
<code>API_PORT</code>	HTTP port (default 4000)
<code>API_CORS_ORIGINS</code>	Allowed CORS origins
<code>AWS_S3_BUCKETS</code>	S3 bucket for media (optional in dev)
<code>AWS_REGION</code>	AWS region
<code>STRIPE_SECRET_KEY</code>	Stripe API key
<code>STRIPE_WEBHOOK_SECRET</code>	Webhook signature secret

11.2 Mobile (`apps/mobile/.env`)

Variable	Purpose
<code>__TABLE_SEP__</code>	EXPO_PUBLIC_API_URL API base URL (use LAN IP for physical device testing) Example: <code>EXPO_PUBLIC_API_URL=http://192.168.4.42:4000/api/v1</code>

11.3 Web (`apps/web/.env.local`)

Variable	Purpose
<code>__TABLE_SEP__</code>	NEXT_PUBLIC_API_URL API base URL for browser requests

12. Database Setup and Seed Data

```
` bash
```

From repository root

```
npm run db:generate # Generate Prisma client
npm run db:migrate # Apply migrations (or prisma db push for dev)
npm run db:seed # Insert demo data`
```

Demo Accounts (password: **Password123!**)

Email	Role
<code>__TABLE_SEP__</code>	homeowner@example.com HOMEOWNER
<code>contractor@example.com</code>	CONTRACTOR
<code>admin@example.com</code>	ADMIN

Seed data includes sample jobs in NYC metro area (ZIP codes 10013, 10002, 11238) with realistic addresses, budgets, and a contractor profile with multiple service types.

13. Local Development Guide

13.1 Prerequisites

1. Node.js ≥ 20
1. PostgreSQL (Neon cloud or local)
1. iOS: Expo Go app on device (same Wi-Fi as dev machine)

13.2 Start All Services

```
` bash
```

Terminal 1 – API

```
npm run api
```

Terminal 2 – Web

```
npm run web
```

Terminal 3 – Mobile (Expo Go)

```
npm run mobile
```

Or: `cd apps/mobile && npm run start:go`

```
`
```

13.3 Mobile Device Testing

Physical devices cannot reach `localhost`. Set `EXPO_PUBLIC_API_URL` to your machine's LAN IP: `EXPO_PUBLIC_API_URL=http://:4000/api/v1` Find your IP with ipconfig (Windows) or ifconfig (macOS/Linux). Reload the app in Expo Go after changing .env.`

13.4 EAS Builds (Production Mobile)

The mobile app includes `eas.json` for Expo Application Services builds. Custom app icons and native modules require an EAS dev build rather than Expo Go. Configuration targets iOS and Android with `expo-dev-client`.

14. Deployment Considerations

ComponentRecommended Target __TABLE_SEP__ APIRailway, Render, Fly.io, or AWS ECS WebVercel or similar Next.js host DatabaseNeon PostgreSQL (already configured) MediaAWS S3 with CloudFront CDN

MobileEAS Build → App Store / Google Play **CI recommendations:** Run `npm run typecheck` , `npm run lint` , and `npm run test` on pull requests. Apply Prisma migrations in the deployment pipeline before starting the API.

15. Known Limitations and Roadmap

AreaCurrent StatePlanned __TABLE_SEP__ GeospatialHaversine on Float columnsPostGIS ST_DWithin OAuthSchema supports Google/AppleUI not implemented RedisNot yet integratedSession cache, rate limiting RatingsSchema fields existReview submission flow Push notificationsDevice token storageFCM/APNs delivery service

16. Document Revision History

DateChange __TABLE_SEP__ 2026-06-07Initial build specification with ER diagram

Generated from the ContractorBidder monorepo source tree. For the latest schema, see `apps/api/prisma/schema.prisma`.`